IF25-22014 PENGEMBANGAN APLIKASI WEB

Pertemuan 10
OOP di Python

Memodelkan data kampus sebagai object, bekal langsung untuk model FastAPI

Tim Pengajar

Tahun 2026. Rujukan: Dokumentasi resmi Python, *Classes* dan *abc*; dokumentasi Pydantic; Algifari dkk., ITERA Press, 2024.

ITERA

DIKTISAINTEK BERDAMPAK

Program Studi Teknik Informatika
Institut Teknologi Sumatera

PETA PERTEMUAN

Perjalanan kalian hari ini

ITERA

Fase	Menit	Yang kalian lakukan	Pertanyaan wajar	Bila tersendat
Pembuka	0–10	Menjawab tiga soal minggu lalu dengan jari	Masih ingat yang kemarin?	Catat nomor yang salah, cek di modul minggu lalu
Coba dulu	10–30	Mengerjakan tiga misi berpasangan	Boleh salah?	Tulis dugaan; salah itu bahan belajar
Materi	30–85	Mengikuti 8 bagian: tebak, ikuti bersama, engsel	Sedang di bagian mana?	Lihat penanda di atas judul atau klik Outline
Hands-on	85–120	Mengerjakan tingkat 1 sampai 3 dari `starter/`	Sudah benar belum?	Periksa berpasangan; buka modul bagian yang sama
Papan	120–140	Menunjukkan atau menanggapi hasil teman	Kenapa kode itu jalan?	Ajukan satu pertanyaan mengapa
Exit ticket	140–150	Menjawab dua soal tanpa catatan	Apa yang belum paham?	Tulis di kolom refleksi

Penanda fase dan bagian ada di atas setiap judul slide. Kotak **Ikuti bersama** berarti ketik di laptop kalian sekarang.

Soal dari Pertemuan 9

Soal 1. Hasil `[n * 2 for n in [1, 2, 3] if n > 1]`?

- **A** [2, 4, 6]
- **B** [4, 6]
- **C** [2, 3]
- **D** [1, 2, 3]

Soal 2. Akses nama dari dict `mhs`?

- **A** `mhs.nama`
- **B** `mhs["nama"]`
- **C** `mhs→nama`
- **D** `mhs(nama)`

Soal 3. Untuk apa `requirements.txt`?

- **A** menyimpan kode
- **B** daftar pustaka proyek
- **C** konfigurasi git
- **D** data JSON

KUNCI

1B: filter dulu, lalu kali. 2B: dict memakai kurung siku. 3B: setara package.json.

Dict yang tidak bisa menjaga dirinya

ITERA

Data mahasiswa disimpan sebagai dict. Suatu hari seseorang menulis kode di samping, dan IPK seluruh angkatan menjadi kacau di laporan.
Dict menerima nilai apa pun dengan kunci apa pun, termasuk salah ketik.

PIKIRKAN

Bagaimana membuat data mahasiswa yang menolak nilai tidak masuk akal dan salah ketik nama kunci?

```
mhs = {"nim": "124140127", "nama": "Rani"}
mhs["nilai"] = 150 # tidak masuk akal
mhs["nialai"] = 80 # salah ketik
mhs["sks"] = "dua puluh" # tipe salah
print(mhs) # semua diterima
```

FASE 2 DARI 6: COBA DULU

Misi kalian, 20 menit

ITERA

Berdua atau bertiga, pakai apa saja yang kalian tahu. Berkas awal ada di folder hands-on, `starter/`.

1 Cetakan

Di `starter/akademik.py`, buat class `Mahasiswa` yang bisa dipakai seperti `Mahasiswa("124140127", "Rani")`.

2 Jaga nilai

Cari cara agar `mhs.nilai = 150` ditolak dengan pesan yang jelas.

3 Satu panggilan

Buat dua class berbeda yang sama-sama punya method `menu()`, lalu panggil keduanya dalam satu perulangan.

Boleh

Bertanya ke teman, mencoba di browser, salah.

Belum dulu

Mencari di internet atau bertanya ke AI.

Delapan bagian hari ini

ITERA

	Bagian	Tujuan (klik judul untuk melompat)
1	**Pengantar OOP**	**Menjelaskan paradigma berorientasi object**
2	Class dan object	Membuat class dan object di Python
3	Atribut dan method	Membedakan atribut object, atribut class, dan method
4	Konstruktor dan self	Menjelaskan peran __init__ dan self
5	Inheritance	Mewariskan atribut dan method ke class turunan
6	Encapsulation	Melindungi data dengan konvensi akses dan property
7	Polymorphism	Memanggil method yang sama pada object berbeda
8	Abstract class dan interface	Membuat template wajib untuk class turunan, dan mengenal model data modern

Data dan perilaku dalam satu tempat

ITERA

Pemrograman berorientasi object (OOP)
Paradigma pemrograman yang memodelkan program sebagai kumpulan object; setiap object menyatukan data (atribut) dan perilaku (method).

Dengan dict, data mahasiswa dan fungsi `ipk` terpisah dan siapa pun bisa mengubah data seenaknya. Dengan OOP, mahasiswa adalah object yang tahu cara menghitung IPK-nya sendiri dan menolak data yang salah. Empat pilar yang dibahas hari ini: encapsulation, inheritance, polymorphism, dan abstraction.

Delapan bagian hari ini

ITERA

	Bagian	Tujuan, dan inti bagian yang sudah dibahas
1	Pengantar OOP	Inti: OOP menyatukan data (atribut) dan perilaku (method) dalam object.
2	**Class dan object**	**Membuat class dan object di Python**
3	Atribut dan method	Membedakan atribut object, atribut class, dan method
4	Konstruktor dan self	Menjelaskan peran __init__ dan self
5	Inheritance	Mewariskan atribut dan method ke class turunan
6	Encapsulation	Melindungi data dengan konvensi akses dan property
7	Polymorphism	Memanggil method yang sama pada object berbeda
8	Abstract class dan interface	Membuat template wajib untuk class turunan, dan mengenal model data modern

Class di JavaScript dan Python

ITERA

JavaScript (P6)

```
class Mahasiswa {
  constructor(nim, nama) {
    this.nim = nim;
    this.nama = nama;
  }
  sapa() { return `Halo ${this.nama}`; }
}
const r = new Mahasiswa("127", "Rani");
```

Python

```
class Mahasiswa:
    def __init__(self, nim, nama):
        self.nim = nim
        self.nama = nama

    def sapa(self):
        return f"Halo {self.nama}"

r = Mahasiswa("127", "Rani")
```

Bedanya

Idenya sama dengan P6. Bedanya: `__init__` bukan `constructor`, `self` ditulis eksplisit sebagai parameter pertama, dan tidak ada kata `new`.

Ikuti bersama: class pertama di Python interaktif

ITERA

1. Buka Python interaktif di terminal.
   ```
   python
   ```
2. Ketik kepala class, lalu Enter.
   ```
   class Mhs:
   ```
3. Ketik konstruktor dengan empat spasi di depan, lalu Enter dua kali.
   ```
       def __init__(self, n): self.nama = n
   ```
4. Buat object dan baca atributnya.
   ```
   r = Mhs("Rani"); r.nama
   ```
5. Coba buat method tanpa `self`, lalu panggil.

YANG HARUS TERLIHAT

Python menampilkan `'Rani'`. Method tanpa `self` memunculkan TypeError yang menyebut jumlah argumen.

Kalau berbeda

IndentationError: setiap baris di dalam class berawal empat spasi.
NameError: nama class harus ditulis sama persis, termasuk huruf besar.

Delapan bagian hari ini

ITERA

	Bagian	Tujuan, dan inti bagian yang sudah dibahas
1	Pengantar OOP	**Inti:** OOP menyatukan data (atribut) dan perilaku (method) dalam object.
2	Class dan object	**Inti:** class cetakan, object hasilnya; setiap object punya data sendiri.
3	**Atribut dan method**	**Membedakan atribut object, atribut class, dan method**
4	Konstruktor dan self	Menjelaskan peran __init__ dan self
5	Inheritance	Mewariskan atribut dan method ke class turunan
6	Encapsulation	Melindungi data dengan konvensi akses dan property
7	Polymorphism	Memanggil method yang sama pada object berbeda
8	Abstract class dan interface	Membuat template wajib untuk class turunan, dan mengenal model data modern

Atribut dan method

ITERA

```
class Mahasiswa:
    kampus = "ITERA" # atribut class

    def __init__(self, nim: str, nama: str):
        self.nim = nim # atribut object
        self.nama = nama
        self.khs: list[dict] = []

    def tambah_nilai(self, matkul: str, sks: int, huruf: str):
        self.khs.append({"matkul": matkul, "sks": sks, "huruf":
huruf})

    def total_sks(self) -> int:
        return sum(m["sks"] for m in self.khs)

r = Mahasiswa("124140127", "Rani")
r.tambah_nilai("Pemweb", 3, "A"); print(r.total_sks())
```

Akses dengan titik: `r.total_sks()`. Bandingkan dengan dict yang memakai kurung siku.

Delapan bagian hari ini

ITERA

	Bagian	Tujuan, dan inti bagian yang sudah dibahas
1	Pengantar OOP	**Inti:** OOP menyatukan data (atribut) dan perilaku (method) dalam object.
2	Class dan object	**Inti:** class cetakan, object hasilnya; setiap object punya data sendiri.
3	Atribut dan method	**Inti:** Atribut object milik sendiri, atribut class dibagi; method bekerja pada self.
4	**Konstruktor dan self**	**Menjelaskan peran __init__ dan self**
5	Inheritance	Mewariskan atribut dan method ke class turunan
6	Encapsulation	Melindungi data dengan konvensi akses dan property
7	Polymorphism	Memanggil method yang sama pada object berbeda
8	Abstract class dan interface	Membuat template wajib untuk class turunan, dan mengenal model data modern

Siapa self?

ITERA

TEBAK DULU

Saat menulis `r.total_sks()`, method `total_sks(self)` dipanggil tanpa argumen. Dari mana `self` datang, dan apa isinya?

JAWABAN

Python mengubah `r.total_sks()` menjadi `Mahasiswa.total_sks(r)`. Jadi `self` adalah `r`, object yang sedang dipakai. Itu sebabnya `self` wajib ditulis sebagai parameter pertama setiap method, dan atribut selalu diakses lewat `self.nama`, bukan `nama` saja.

Validasi di konstruktor

```
class Mahasiswa:
    def __init__(self, nim: str, nama: str):
        if not (nim.isdigit() and len(nim) == 9):
            raise ValueError("NIM harus 9 digit angka")
        if not nama.strip():
            raise ValueError("Nama tidak boleh kosong")
        self.nim = nim
        self.nama = nama.strip()

Mahasiswa("12AB", "Rani")
# ValueError: NIM harus 9 digit angka
```

Inilah bagian pertama jawaban masalah minggu ini. FastAPI di P11 melakukan validasi semacam ini otomatis dari type hints.

Delapan bagian hari ini

ITERA

	Bagian	Tujuan, dan inti bagian yang sudah dibahas
1	Pengantar OOP	**Inti:** OOP menyatukan data (atribut) dan perilaku (method) dalam object.
2	Class dan object	**Inti:** class cetakan, object hasilnya; setiap object punya data sendiri.
3	Atribut dan method	**Inti:** Atribut object milik sendiri, atribut class dibagi; method bekerja pada self.
4	Konstruktor dan self	**Inti:** __init__ dijalankan saat object dibuat untuk validasi awal; self object yang sedang dipakai.
5	**Inheritance**	**Mewariskan atribut dan method ke class turunan**
6	Encapsulation	Melindungi data dengan konvensi akses dan property
7	Polymorphism	Memanggil method yang sama pada object berbeda
8	Abstract class dan interface	Membuat template wajib untuk class turunan, dan mengenal model data modern

Pewarisan

ITERA

```
class Pengguna:
    def __init__(self, nama: str, email: str):
        self.nama = nama
        self.email = email

    def menu(self) → list[str]:
        return ["Beranda"]

class Mahasiswa(Pengguna):
    def __init__(self, nama: str, email: str, nim: str):
        super().__init__(nama, email)
        self.nim = nim

    def menu(self) → list[str]:
        return super().menu() + ["KRS", "KHS"]
```

Di P11 kalian menulis `class Matkul(BaseModel)`: mewarisi kemampuan validasi dari Pydantic.

Delapan bagian hari ini

ITERA

	Bagian	Tujuan, dan inti bagian yang sudah dibahas
1	Pengantar OOP	**Inti:** OOP menyatukan data (atribut) dan perilaku (method) dalam object.
2	Class dan object	**Inti:** class cetakan, object hasilnya; setiap object punya data sendiri.
3	Atribut dan method	**Inti:** Atribut object milik sendiri, atribut class dibagi; method bekerja pada self.
4	Konstruktor dan self	**Inti:** __init__ dijalankan saat object dibuat untuk validasi awal; self object yang sedang dipakai.
5	Inheritance	**Inti:** class Anak(Induk) mewarisi atribut dan method; super() memanggil induk.
6	**Encapsulation**	**Melindungi data dengan konvensi akses dan property**
7	Polymorphism	Memanggil method yang sama pada object berbeda
8	Abstract class dan interface	Membuat template wajib untuk class turunan, dan mengenal model data modern

Konvensi akses dan property

ITERA

```
class NilaiMatkul:
    def __init__(self, matkul: str, nilai: float):
        self.matkul = matkul
        self.nilai = nilai # lewat setter di bawah

    @property
    def nilai(self) → float:
        return self._nilai

    @nilai.setter
    def nilai(self, n: float):
        if not 0 ⩽ n ⩽ 100:
            raise ValueError("Nilai harus 0 sampai 100")
        self._nilai = n

n = NilaiMatkul("Pemweb", 86)
n.nilai = 150 # ValueError
```

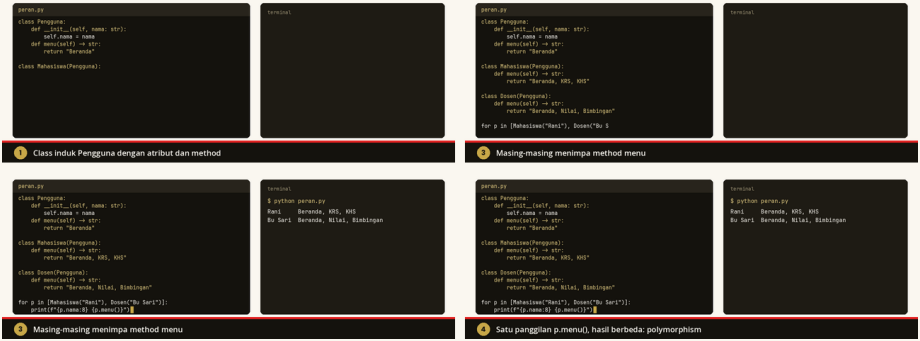
Bagian kedua jawaban masalah minggu ini: nilai 150 ditolak.

Delapan bagian hari ini

ITERA

	Bagian	Tujuan, dan inti bagian yang sudah dibahas
1	Pengantar OOP	**Inti:** OOP menyatukan data (atribut) dan perilaku (method) dalam object.
2	Class dan object	**Inti:** class cetakan, object hasilnya; setiap object punya data sendiri.
3	Atribut dan method	**Inti:** Atribut object milik sendiri, atribut class dibagi; method bekerja pada self.
4	Konstruktor dan self	**Inti:** __init__ dijalankan saat object dibuat untuk validasi awal; self object yang sedang dipakai.
5	Inheritance	**Inti:** class Anak(Induk) mewarisi atribut dan method; super() memanggil induk.
6	Encapsulation	**Inti:** Garis bawah menandai internal; @property dan setter menjaga nilai.
7	**Polymorphism**	**Memanggil method yang sama pada object berbeda**
8	Abstract class dan interface	Membuat template wajib untuk class turunan, dan mengenal model data modern

Satu panggilan, banyak perilaku



Perulangan hanya memanggil p.menu(); setiap jenis pengguna menjawab dengan menunya sendiri. Ini jawaban misi 3.

Hasil overriding

ITERA

Dengan class di animasi tadi, apa hasil `Pengguna("Tamu").menu()` dan `isinstance(Mahasiswa("R"), Pengguna)`?

- **A** Beranda dan False
- **B** Beranda dan True
- **C** KRS dan True
- **D** error dan True

JAWABAN: B

Pengguna memakai method miliknya sendiri. Mahasiswa adalah turunan Pengguna, jadi `isinstance` bernilai True.

Delapan bagian hari ini

ITERA

	Bagian	Tujuan, dan inti bagian yang sudah dibahas
1	Pengantar OOP	**Inti:** OOP menyatukan data (atribut) dan perilaku (method) dalam object.
2	Class dan object	**Inti:** class cetakan, object hasilnya; setiap object punya data sendiri.
3	Atribut dan method	**Inti:** Atribut object milik sendiri, atribut class dibagi; method bekerja pada self.
4	Konstruktor dan self	**Inti:** __init__ dijalankan saat object dibuat untuk validasi awal; self object yang sedang dipakai.
5	Inheritance	**Inti:** class Anak(Induk) mewarisi atribut dan method; super() memanggil induk.
6	Encapsulation	**Inti:** Garis bawah menandai internal; @property dan setter menjaga nilai.
7	Polymorphism	**Inti:** Method sama, perilaku berbeda per object; pemanggil tak perlu tahu jenisnya.
8	**Abstract class dan interface**	**Membuat template wajib untuk class turunan, dan mengenal model data modern**

Abstract class

```
from abc import ABC, abstractmethod

class Penilaian(ABC):
    def __init__(self, bobot: float):
        self.bobot = bobot

    @abstractmethod
    def skor(self) → float: ...
class Kuis(Penilaian):
    def __init__(self, bobot: float, benar: int, total:
int):
        super().__init__(bobot)
        self.benar, self.total = benar, total
    def skor(self) → float:
        return 100 * self.benar / self.total

Penilaian(0.2) # TypeError: tidak bisa dibuat langsung
```

Kode yang memakai `Penilaian` cukup memanggil `skor()` untuk kuis, tugas, atau ujian.

Class data siap pakai

ITERA

dataclass (bawaan Python)

```
from dataclasses import dataclass

@dataclass
class Matkul:
    kode: str
    nama: str
    sks: int = 3

m = Matkul("IF25-22014", "Pemweb")
```

Pydantic, dipakai FastAPI

```
from pydantic import BaseModel, Field

class Matkul(BaseModel):
    kode: str
    nama: str
    sks: int = Field(3, ge=1, le=6)

Matkul(kode="IF", nama="Web", sks="dua")
# ValidationError: sks harus int
```

Bedanya

Pydantic menambah validasi dan konversi JSON pada class data: jawaban lengkap masalah minggu ini, dan model setiap endpoint P11.

Tiga baris yang salah

ITERA

Yang ditulis	Mengapa salah, dan perbaikannya
`def total_sks():` di dalam class	Lupa `self`; TypeError saat dipanggil. Tulis `def total_sks(self):`.
`khs = []` sebagai atribut class	List terbagi ke semua object; nilai Rani muncul di KHS Bima. Pindahkan ke `self.khs = []` di `__init__`.
`super.__init__(nama)`	`super` harus dipanggil: `super().__init__(nama)`.

Pola kesalahannya

Kesalahan kedua paling berbahaya karena tidak ada error sama sekali; datanya diam-diam tercampur.

Giliran kalian

ITERA

1 **Setengah jadi: tugas sebagai penilaian**

Lengkapi: `class Tugas(_____):` dengan `def skor(_____) → float: return self._____` untuk nilai tugas 0 sampai 100.

2 **Mandiri: dosen**

Tulis `class Dosen(Pengguna)` dengan atribut `nip` 18 digit yang divalidasi di konstruktor, dan `menu()` yang menambah *Nilai* dan *Bimbingan* ke menu induk.

JAWABAN

Setengah jadi: Penilaian, self, nilai. Mandiri: super().__init__(nama, email); if not (nip.isdigit() and len(nip) == 18): raise ValueError(...); menu mengembalikan super().menu() + [”Nilai”, ”Bimbingan”].

Hands-on bertingkat

ITERA

Tingkat	Waktu	Yang dilatih	Sub-CPMK
Tingkat 1, Dasar	10′	class, konstruktor dengan validasi, atribut, dan method	10.1
Tingkat 2, Menengah	15′	encapsulation dengan property, inheritance, dan polymorphism	10.2
Tingkat 3, Tantangan	10′	abstract class dan perhitungan nilai akhir	10.2
Pengayaan	sisa	model Pydantic sebagai persiapan P11	10.2

Aturan

Berkas awal ada di `02 Hands-on/P10 - OOP Python - Hands-on/starter`. Kerjakan berurutan dari tingkat 1, simpan bukti tangkapan layar, dan sebutkan alasan setiap langkah.

Presentasi di papan

ITERA

Tiga mahasiswa diundi untuk menunjukkan hasil hands-on tingkat 2 atau 3 di layar, masing-masing sekitar 5 menit, lalu menjawab satu pertanyaan.

Kriteria	Sangat baik 75--100	Baik 65--74	Cukup 55--64	Kurang <55
Ketepatan hasil	berjalan dan sesuai soal	satu salah kecil	ada bagian salah	sebagian besar keliru
Alasan tiap langkah	setiap baris dijelaskan fungsinya	sebagian besar dijelaskan	penjelasan kabur	tanpa penjelasan
Menjawab pertanyaan	tanpa melihat catatan	dengan sedikit bantuan	menjawab sebagian	tidak bisa menjawab

Yang tidak maju tetap bisa menambah nilai lewat pertanyaan atau koreksi yang tepat.

Exit ticket Pertemuan 10

ITERA

```
class Akun:
    saldo_awal = []
    def tambah(nilai):
        self.saldo_awal.append(nilai)
```

1 Sub-CPMK 10.1

Kelas di atas punya dua kesalahan. Sebutkan dan tulis versi yang benar.

2 Sub-CPMK 10.2

Tulis `class Asisten(Mahasiswa)` yang menambah atribut `matkul` dan menimpa `menu()` agar menambah *Praktikum* ke menu induk, dengan *A* dipakai sebagai nomor kelas praktikum.

3 Refleksi

Tulis satu kalimat: bagian mana yang paling membingungkan hari ini, dan mengapa?

Soal 1 dan 2 dinilai 0, 1, atau 2: benar dengan alasan, benar tanpa alasan atau salah kecil, belum benar.

Yang dibawa pulang

ITERA

	Bagian	Inti
1	Pengantar OOP	OOP menyatukan data (atribut) dan perilaku (method) dalam object.
2	Class dan object	class cetakan, object hasilnya; setiap object punya data sendiri.
3	Atribut dan method	Atribut object milik sendiri, atribut class dibagi; method bekerja pada self.
4	Konstruktor dan self	__init__ dijalankan saat object dibuat untuk validasi awal; self object yang sedang dipakai.
5	Inheritance	class Anak(Induk) mewarisi atribut dan method; super() memanggil induk.
6	Encapsulation	Garis bawah menandai internal; @property dan setter menjaga nilai.
7	Polymorphism	Method sama, perilaku berbeda per object; pemanggil tak perlu tahu jenisnya.
8	Abstract class dan interface	abstractmethod memaksa turunan; dataclass dan Pydantic BaseModel class data siap pakai.



Ada pertanyaan?

Sampai jumpa di Pertemuan 11: Kerangka Kerja FastAPI